Université de Lorraine
Imed KACEM
UFR MIM / LCOMS

TD Algorithmique avancée : PTAS et FPTAS

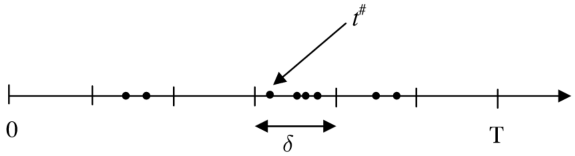
On considère le problème **simple** de **sac-à-dos (SSD)** qui consiste à choisir un sous-ensemble d'objets parmi N objets indépendants (i=1,2,3…N). Ce sous-ensemble doit pouvoir rentrer dans un sac de volume T. Chaque objet i possède un volume p_i et un profit p_i (les deux valeurs sont identiques). L'objectif est de trouver un sous-ensemble d'objets permettant de maximiser le profit total des objets sélectionnés en respectant la contrainte de capacité T du sac.

1) Donner la taille d'une instance de ce problème (nombre de bits nécessaires pour coder toutes ses données).
2) On considère l'heuristique H consistant à insérer (tant que l'on peut) les objets dans l'ordre décroissant des p_i (règle *LPT* classique). Montrer que H ≥ OPT/2 (avec OPT la valeur de la solution optimale).
   Vérifier la relation pour l'instance à 3 objets : p_1 =1001, p_2 =1000, p_3 =1000 et T=2000. Pour ce faire, calculer H et OPT pour cette instance.
3) Soit ε > 0. L'objectif de cette question est de montrer que l'on peut élaborer un PTAS pour le problème SSD. Rappel : PTAS = Polynomial Time Approximation Scheme – algorithme de complexité polynomiale en fonction de la taille de l'instance pour une valeur ε fixée, avec ε la précision souhaitée, tel que la solution retournée S_{PTAS} vérifie $S_{PTAS} \geq (1 - \varepsilon).OPT$. **Attention, c'est un problème de maximisation.** Soit $G=\{i \in \{1,2,\ldots,N\} \mid p_i \geq 2\varepsilon H\}$ l'ensemble des grands objets. Les objets restants constituent le sous-ensemble S et sont appelés les petits objets.
   - Une solution optimale peut contenir combien d'objets de G ?
   - Montrer que le nombre d'affectations de G est inférieur à $N^{(1/\varepsilon)}/\varepsilon$.
   - Déduire un algorithme PTAS en s'inspirant des exemples vus en cours.
4) En observant que la solution optimale est une partition d'objets (en deux sous-ensembles), proposer un algorithme de programmation dynamique *A* pour résoudre ce problème.
   Indications : Essayer de vous inspirer des exemples vus en cours. On pourra prendre comme états générés par l'algorithme des vecteurs [*t*] (à une seule composante), en sachant que *t* représente la somme des profits (volumes) des objets sélectionnés.
5) On peut remarquer que *t* est inférieur ou égal à T pour toute solution réalisable.
   Expliquer comment on peut éventuellement améliorer la complexité de la programmation dynamique.
6) En simplifiant la programmation dynamique montrer que le problème admet un FPTAS. Rappel : FPTAS = Fully Polynomial Time Approximation Scheme – algorithme PTAS de complexité polynomiale en fonction de la taille de l'instance et de 1/ε.
   Indications : Il faudrait considérer des états approchés [$t^{\#}$] générés par une version tronquée de l'algorithme de programmation dynamique, appelée $A^{\#}$. On pourra montrer qu'à chaque itération *i* de ces algorithmes, un état [*t*] généré par *A* possède un état homologue [$t^{\#}$] généré par $A^{\#}$ avec la condition suivante :

$$t - i.\delta \leq t^{\#} \leq t$$

   avec $\delta = \varepsilon T/N$.

   Dans la phase de remplacement des états [*t*] par l'état [$t^{\#}$], on prendra toujours l'état avec le $t^{\#}$ plus petit dans les petites zones (segments de dimension identique δ) de l'espace d'états (ici, l'espace d'états est un segment car chaque état contient une seule composante).



1) Donner la taille d'une instance de ce problème (nombre de bits nécessaires pour coder toutes ses données).

1- Une instance du problème peut être :

$$I = \langle N, T, p_1, p_2, \dots, p_{n-1}, p_n \rangle.$$

En binaire, la taille $|I|_2 = \log_2(N) + \log_2(T) + \sum_{i=1}^{N} \log_2(p_i)$.

$\forall i, \exists p_{max}$ tq $p_i \leq p_{max}$.

Donc la taille $|I|_2 \leq \log_2(N) + \log_2(T) + N \log_2(p_{max})$

En unaire, la taille $|I|_1 = N + T + \sum p_i$
$\leq N + T + p_{max}$.

2) On considère l'heuristique H consistant à insérer (tant que l'on peut) les objets dans l'ordre décroissant des p_i (règle *LPT* classique). Montrer que H ≥ OPT/2 (avec OPT la valeur de la solution optimale).
Vérifier la relation pour l'instance à 3 objets : p_1 =1001, p_2 =1000, p_3 =1000 et T=2000. Pour ce faire, calculer H et OPT pour cette instance.

Pour $I = \langle 3, 2000, 1001, 1000, 1000 \rangle$, LPT = décroissant des p_i.



et 2 ≠ 3 pas placables.

De manière générale,



δ : espace non rempli.

et $H = T - \delta$.

Lemme : $\delta \leq \frac{T}{2}$.

Optimale : $\delta = 0$ et $H = T = OPT$.

Pire des cas : $p_1 : \frac{T}{2} + 1$ et $\forall i \geq 2,\ p_i = T$. Seulement p_1 est plaçable donc $H = \frac{T}{2} + 1$ et $\delta = \frac{T}{2} - 1$.

Par l'absurde : si $\delta \geq \frac{T}{2}$!

$$p_1 \geq p_2 > \delta \geq \frac{T}{2}$$

LPT — tous les $p_i > \delta$ sinon tâche plaçable.

$p_1 + \delta > T$ impossible, donc en fait on ne peut rien placer dans ce cas. Impossible.

Donc $\delta < \frac{T}{2}$.

$$\Rightarrow H = T - \delta \geq \frac{T}{2} \geq \frac{OPT}{2}.$$

3) Soit $\varepsilon > 0$. L'objectif de cette question est de montrer que l'on peut élaborer un PTAS pour le problème SSD. Rappel : PTAS = Polynomial Time Approximation Scheme – algorithme de complexité polynomiale en fonction de la taille de l'instance pour une valeur ε fixée, avec ε la précision souhaitée, tel que la solution retournée S_{PTAS} vérifie $S_{PTAS} \geq (1 - \varepsilon).OPT$. **Attention, c'est un problème de maximisation.** Soit $G=\{i \in \{1,2,\ldots,N\} \mid p_i \geq 2\varepsilon H\}$ l'ensemble des grands objets. Les objets restants constituent le sous-ensemble S et sont appelés les petits objets.

- Une solution optimale peut contenir combien d'objets de G ?
- Montrer que le nombre d'affectations de G est inférieur à $N^{(1/\varepsilon)}/\varepsilon$.
- Déduire un algorithme PTAS en s'inspirant des exemples vus en cours.

PTAS	FPTAS
poly(|I|)	poly(|I|) + poly($\frac{1}{\varepsilon}$)

$G = \{ i \mid p_i \geq 2\varepsilon H \}$ et $S = \Omega \setminus G$.

- La somme de tous les objets de la solution optimale est OPT, $|G| \leq \frac{OPT}{2\varepsilon H}$ ← taille totale / ← taille partielle.

avec $H \geq \frac{OPT}{2} \Rightarrow 2H \geq OPT$.

$1+\varepsilon$ si min

$1-\varepsilon$ si max

donc $|G| \leqslant \frac{OPT}{\ell \in H} \leqslant \frac{\ell H}{\ell \in H} = \frac{1}{\varepsilon}$.

2. Il faut quantifier toutes les permutat°s de G : $|G| = \frac{1}{\varepsilon}$

1 grad obj. $\binom{n}{1}$ — 2 grands objs $\binom{n}{2}$ — 3 grands objs $\binom{n}{3}$ — ... — $\frac{1}{\varepsilon}$ grands objs $\binom{n}{1/\varepsilon}$

Ce qui donne $\sum_{p=1}^{1/\varepsilon} \binom{n}{p} \leqslant \sum_{p=1}^{1/\varepsilon} n^p \leqslant \sum_{p=1}^{1/\varepsilon} n^{1/\varepsilon} = \frac{1}{\varepsilon}\left(n^{1/\varepsilon}\right) = O\left(\frac{1}{\varepsilon} n^{1/\varepsilon}\right)$

on admet l'ordre de grandeur. — borrage, $n \geq 0$ et $n^{1/\varepsilon} \geqslant n^p$ $\forall p \in [\![1, 1/\varepsilon]\!]$

3. Algo :

1). Diviser les objets : $G = \{ i \mid p_i \geqslant \ell \in H \}$ $\quad O(n)$
$S = \{ i \mid p_i < \ell \in H \}$.

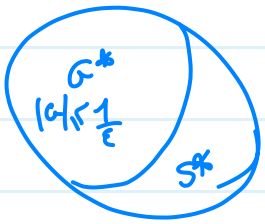
2). Énumérer toutes les affectat°s réalisables de G. $\quad O\left(\frac{1}{\varepsilon} n^{1/\varepsilon}\right)$

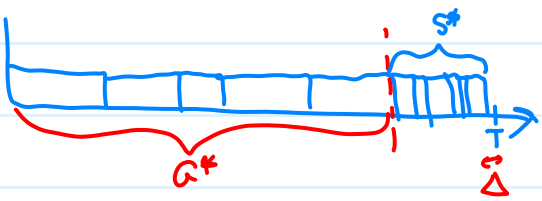
3). Compléter chq affectat° de G générée au EL en insérant les objs de S de n'importe quel ordre tq possible. $\quad O\left(\frac{1}{\varepsilon} n^{1/\varepsilon} \cdot n\right)$ ($\leq$ nb petites tâches)

4). Retrouver la meilleure affectat° $\quad O\left(\frac{1}{\varepsilon} n^{1/\varepsilon}\right)$

→ L'algorithme est en $O\left(\frac{1}{\varepsilon} n^{1/\varepsilon} \cdot n\right)$

4. Analyse :





G^* : grands objets

S^* : petits objets.

Si on suppose que $\sum p_i \geq T$,

$\Delta < \varepsilon T$ car sinon on peut combler avec un petit objet, en supposant que tous les petits objets ont $p_i = \varepsilon T$.

↳ Donc $C = T - \Delta \geq T - \varepsilon T$.

On ne peut pas utiliser $H \geq \frac{OPT}{2}$ car $-\varepsilon H \leq -\frac{OPT}{2} \cdot \varepsilon$

On a $T \geq OPT$ et $C \geq OPT - \varepsilon H$.

On a $H \leq OPT$ et $C \geq OPT - \varepsilon OPT = (1-\varepsilon) OPT$

a facteur 2 est mauvais et évitable si le critère est εH et non pas εT.

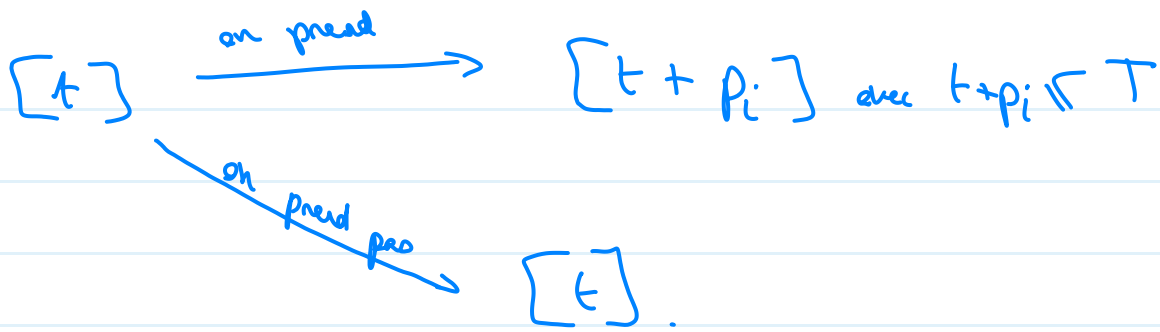
Donc $C \geq (1-\varepsilon) OPT$ donc le rapport est $1-\varepsilon$. Comme on peut choisir ε, on peut prendre $\varepsilon' = \varepsilon$ et avoir $1-\varepsilon'$ équivalent à $1-\varepsilon$ au le choix.

$\frac{C}{OPT} \geq 1-\varepsilon$.

4) En observant que la solution optimale est une partition d'objets (en deux sous-ensembles), proposer un algorithme de programmation dynamique A pour résoudre ce problème.
Indications : Essayer de vous inspirer des exemples vus en cours. On pourra prendre comme états générés par l'algorithme des vecteurs $[t]$ (à une seule composante), en sachant que t représente la somme des profits (volumes) des objets sélectionnés.

4. La décision est : on prend ou on prend pas.

À partir de ce constat, on a :



En algo :

$X_0 = \{[0]\}$.

Pour i allant de 1 à n faire
- $X_i = \{\}$
- Pour chq état $[t] \in X_{i-1}$ faire
  - Ajouter $[t+p_i]$ à X_i si $t+p_i \leqslant T$
  - Ajouter $[t]$ à X_i
- fin
- Supprimer X_{i-1}
- Pour chq valeur de t $(0 \leqslant t \leqslant T)$
  - garder un seul état (on enlève les doublons)
- fin

fin

$O(n \cdot T)$

Retourner $\max \{t\}$
$[t] \in X_n$.

5) On peut remarquer que t est inférieur ou égal à T pour toute solution réalisable. Expliquer comment on peut éventuellement améliorer la complexité de la programmation dynamique.

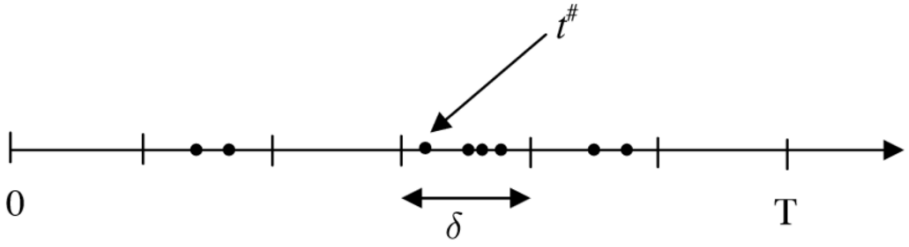
L'objectif est de réduire les valeurs possibles de T.
Pour le faire, on découpe en intervalles et on ne garde que les éléments avec le valeur t minimale => on instaure la dominat°.

6) En simplifiant la programmation dynamique montrer que le problème admet un FPTAS. Rappel : FPTAS = Fully Polynomial Time Approximation Scheme – algorithme PTAS de complexité polynomiale en fonction de la taille de l'instance et de 1/ε.
Indications : Il faudrait considérer des états approchés $[t^{\#}]$ générés par une version tronquée de l'algorithme de programmation dynamique, appelée $A^{\#}$. On pourra montrer qu'à chaque itération i de ces algorithmes, un état $[t]$ généré par A possède un état homologue $[t^{\#}]$ généré par $A^{\#}$ avec la condition suivante :

$$t - i.\delta \leq t^{\#} \leq t$$

avec $\delta = \varepsilon T/N$.

Dans la phase de remplacement des états $[t]$ par l'état $[t^{\#}]$, on prendra toujours l'état avec le $t^{\#}$ plus petit dans les petites zones (segments de dimension identique δ) de l'espace d'états (ici, l'espace d'états est un segment car chaque état contient une seule composante).



$\mathcal{X}_0^\# = \{[0]\}$.

Par i allant de 1 à n faire
  $\mathcal{X}_i^\# = \{\}$
  Par chq état $[t]^\# \in \mathcal{X}_{i-1}$ faire
    Ajouter $[t^\# + p_i]$ à $\mathcal{X}_i^\#$ si $t^\# + p_i \leqslant T$
    Ajouter $[t]^\#$ à $\mathcal{X}_i^\#$
  fin
  Supprimer $\mathcal{X}_{i-1}^\#$
  par chq zone de faille δ faire
    on garde un seul état tq arcmin $\{b\}$ ds l'intervalle.
  fin (enlève les doublons $ET^{[t^\#]}$ dominé en prenant le min sur chq intervalle.)
fin

Retourner $\max \{t^\#\}$
$[t]^\# \in \mathcal{X}_n^\#$.

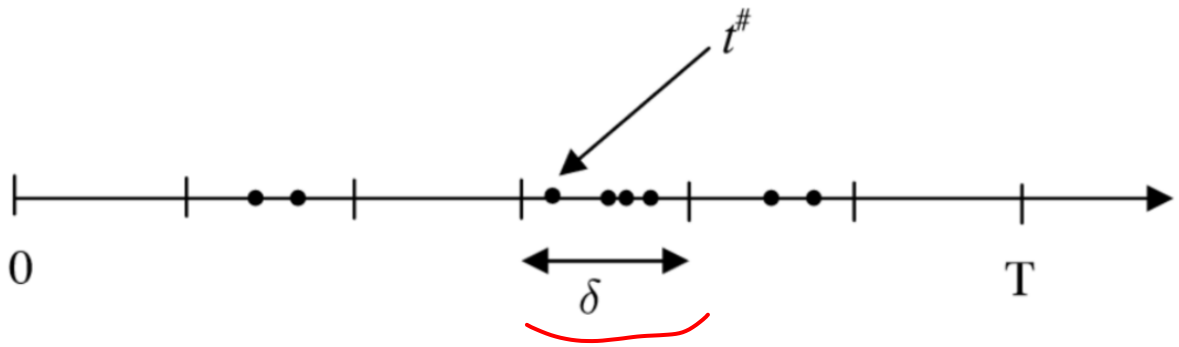
FPTAS

On a $\frac{I}{\delta}$ intervalles. $\frac{I}{\delta} = \frac{I}{\frac{\varepsilon I}{n}} = \frac{n}{\varepsilon}$.

$\Rightarrow \mathcal{O}(nT) \Rightarrow \mathcal{O}\left(n \cdot \frac{n}{\varepsilon}\right) = \mathcal{O}\left(\frac{n^2}{\varepsilon}\right)$.

Lemme : $\forall [t] \in \mathcal{X}_i$, $\exists [t^\#] \in \mathcal{X}_i^\#$ | $t - i\delta \leqslant t^\# \leqslant t$.

→ On perd sans doute l'optimale, mais $t^\#$ est suffisamment proche donc c'est pas grave.



⇒ À chq itération, on s'éloigne progressivement de δ

Par le rapport d'analyse, on suppose le lemme vrai:

L'optimale est $[t^*] \in \chi_n$, et on a $[t^\#_*] \in \chi_n^\#$
Montrer que l'algo est $(1-\varepsilon)$-approximé

Preuve: Soit $[OPT] = [t^*] \in \chi_n$:

En utilisant le lemme, $\exists\, [\bar{t}] \in \chi_n^\# \mid t^* - n\delta \leqslant \bar{t} \leqslant t^*$

$\Rightarrow \bar{t} \geqslant t^* - n\frac{\varepsilon T}{n} = OPT - \varepsilon T$

On a $T \geqslant OPT$ mais l'inégalité n'est pas dans le bon sens:

$$\frac{T}{2} \leqslant OPT$$

Donc: $\bar{t} \geqslant OPT - 2\varepsilon OPT = (1-2\varepsilon)OPT.$

Par ne pas avoir un facteur 2, on dit que δ est paramétrable.
On obtient:

$$\bar{t} \geqslant t^* - n\delta = OPT - n\delta \geqslant \dots \geqslant OPT - \varepsilon OPT = (1-\varepsilon)OPT.$$

La valeur de δ doit permettre de rendre l'inégalité vraie.

Essayons $n\delta = \varepsilon OPT \Rightarrow \delta = \frac{\varepsilon OPT}{n}$.

Mais, le choix de δ doit se faire à l'intérieur de l'algo, et on ne peut pas déterminer OPT à l'avance.

Ayant $OPT \geq \frac{T}{2}$, on aurait :

$$n\delta = \frac{\varepsilon T}{\delta} \Rightarrow \delta = \frac{\varepsilon T}{2n}$$

On obtient $\tilde{t} \geq t^* - n\delta = OPT - n\delta = OPT - \varepsilon\frac{T}{2}$

$\Rightarrow \tilde{t} \geq OPT - \varepsilon OPT = (1-\varepsilon)OPT$

Preuve du lemme : Par récurrence

Init : $i = 0$, $\chi_0 = \{[0]\} = \chi_0^{\#}$: $0 - \delta \leq 0 \leq 0$.

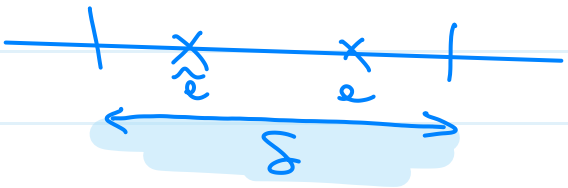
Hér : Vrai au rang $i-1$, vérifions i.

$[t] \in \chi_i$. 2 cas :

Objet pris	Objet pas pris
$[t] = [t' + p_i]$, $t' \in \chi_{i-1}$ (3)	$[t] = [t']$, $t' \in \chi_{i-1}$
Hyp : Comme $[t'] \in \chi_{i-1}$, $\exists [t'^*] \in \chi_{i-1}^{\#}$ tq :	Similaire, à faire à la maison
$t' - (i-1)\delta \leq t'^* \leq t'$ (1) (2)	
L'état (4) $e = [t'^* + p_i]$ est envisagé	

Lors de la construction de $\chi_i^{\#}$ par FPTAS, car $t'^{\#} + p_i \overset{(2)}{\leqslant} t' + p_i = t \overset{(3)}{\leqslant} T$.

e est donc faisable. Toutefois, e peut être remplacé par FPTAS par un autre état $\tilde{e} = (\tilde{t})$ de la même intervalle que e.



On a :

(i) $\tilde{t} \overset{(4)}{\leqslant} t'^{\#} + p_i \overset{(2)}{\leqslant} t' + p_i \overset{(3)}{=} t$

(ii) $(t'^{\#} + p_i) - \tilde{t} \leqslant \delta$

e et e' dans le même intervalle

$$\Rightarrow \tilde{t} \geqslant t'^{\#} + p_i - \delta \overset{(1)}{}$$

$$\tilde{t} \geqslant t' - (i-1)\delta + p_i - \delta$$

$$= t' + p_i - i\delta = t - i\delta \overset{(3)}{}$$